

Apache IoTDB: A Time Series Database for Large Scale IoT Applications

著者：Chen Wang

発表日：2025年5月28日

発表者：ウンダラマー

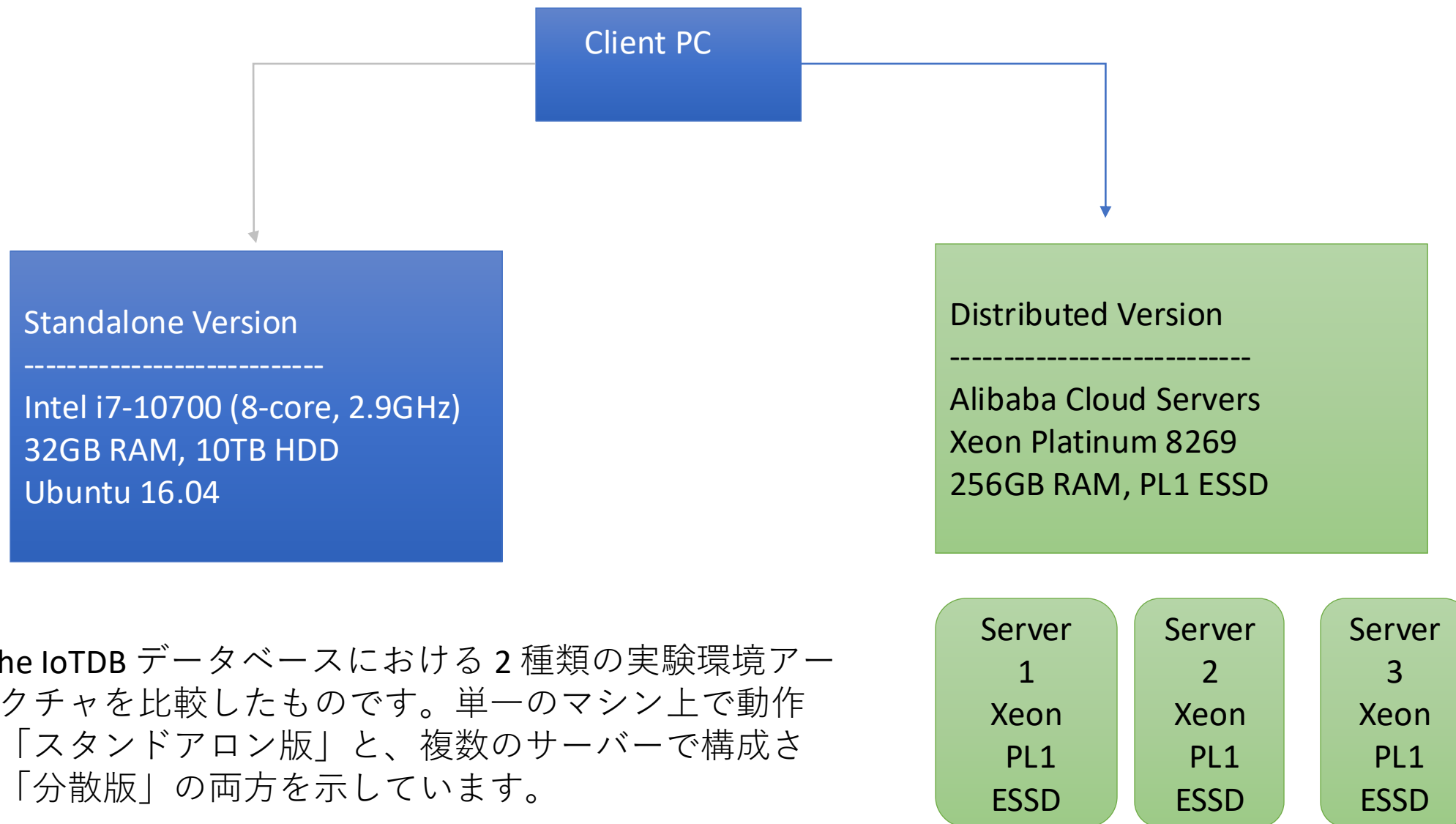
Section 8: Experiments (pp.30-40)

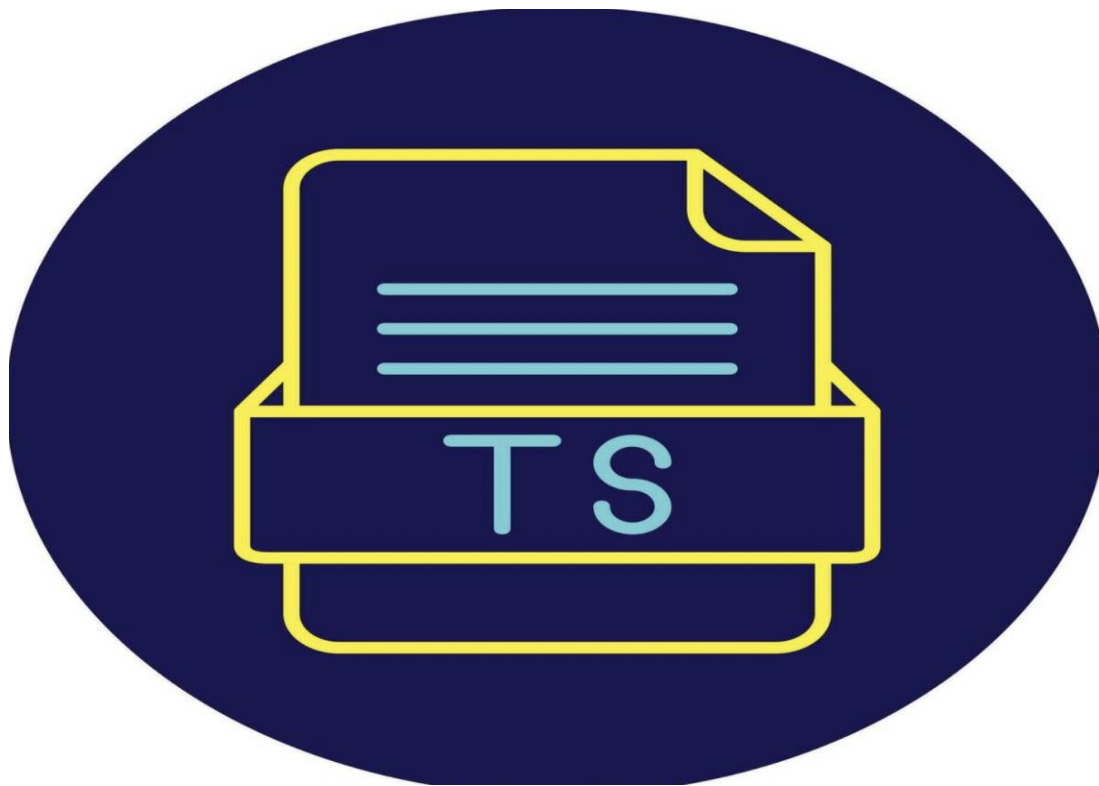
目次

本発表では、大規模なIoTデータ環境におけるApache IoTDBの性能を評価した実験について紹介します.

- IoT 向けに設計された 新しい時系列管理システム
- リアルタイムクエリ と ビッグデータ分析 の両方をサポート
- オープンアーキテクチャ ベースの設計

研究背景





•**役割**：時間系列データ（Time-Series Data）を効率的に保存するための、Apache IoTDBにおける内部ファイルフォーマット。

•**仕組み**：
TsFileは**カラム型ストレージ**（列指向）を採用しており、センサーごとのデータを時間順に圧縮して保存します。これにより、高速なクエリ処理と高圧縮率を実現します。



役割：大規模データをカラム指向形式で保存する高性能フォーマット。主に分析・機械学習などで使用されます。

仕組み：

列ごとにデータを保存するため、必要な列だけを読み込むことができ、I/Oのパフォーマンスが大幅に向上します。Hadoop、Spark、Prestoなどと互換性があります。

Apache ORC (Optimized Row Columnar) v1.5.4



役割：Hadoopエコシステム向けに最適化されたカラム形式のデータ保存フォーマット

仕組み：

Parquetと似たようにカラム単位でデータを格納し、圧縮・インデックス化により高速処理を実現します。HiveやSparkと連携して利用されます



役割：時系列データベース（TSDB）：

IoTデバイス、モニタリング、センサーなどのデータ収集に最適

仕組み：

独自のストレージエンジン（TSM）を使い、時間順にデータを保存します。SQLに似たクエリ言語（InfluxQL）を使用し、集計・分析に優れています

KairosDB Logo



役割： Cassandra上に構築された時系列データベース。

仕組み：

NoSQLであるCassandraのスケラブルな分散型ストレージを活用し、KairosDBは時間系列データをAPI経由で効率的に保存・取得します。大量データや分散環境に適しています。

TimescaleDB v1.7.5



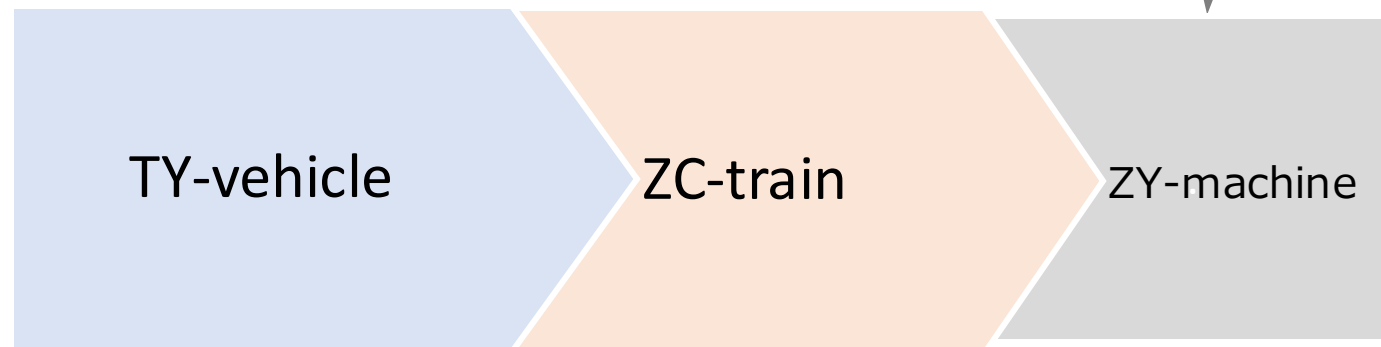
- 役割**：PostgreSQLをベースにした時系列データベース

- 仕組み**：

PostgreSQLの拡張として動作し、時間軸に最適化された「ハイパーテーブル（Hypertable）」を使ってデータを管理します。SQL互換であるため、一般的なデータベース知識で利用可能です

Dataset	# time series	# data points	total size
TY-vehicle	28,781	1,511,347,655	56G
ZC-train	93,284	46,708,284,999	248G
ZY-machine	111,027	17,580,821,079	735G
TSBS	300,000	92,897,280,000	70.8G

産業パートナーから提供された**3つ**
の実データセットと、**1つ**の時系列
ベンチマークデータを使用しました。



TY-vehicle

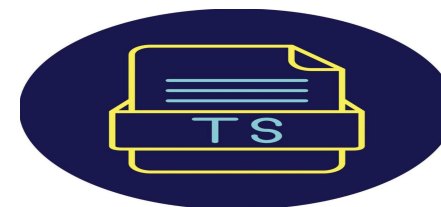
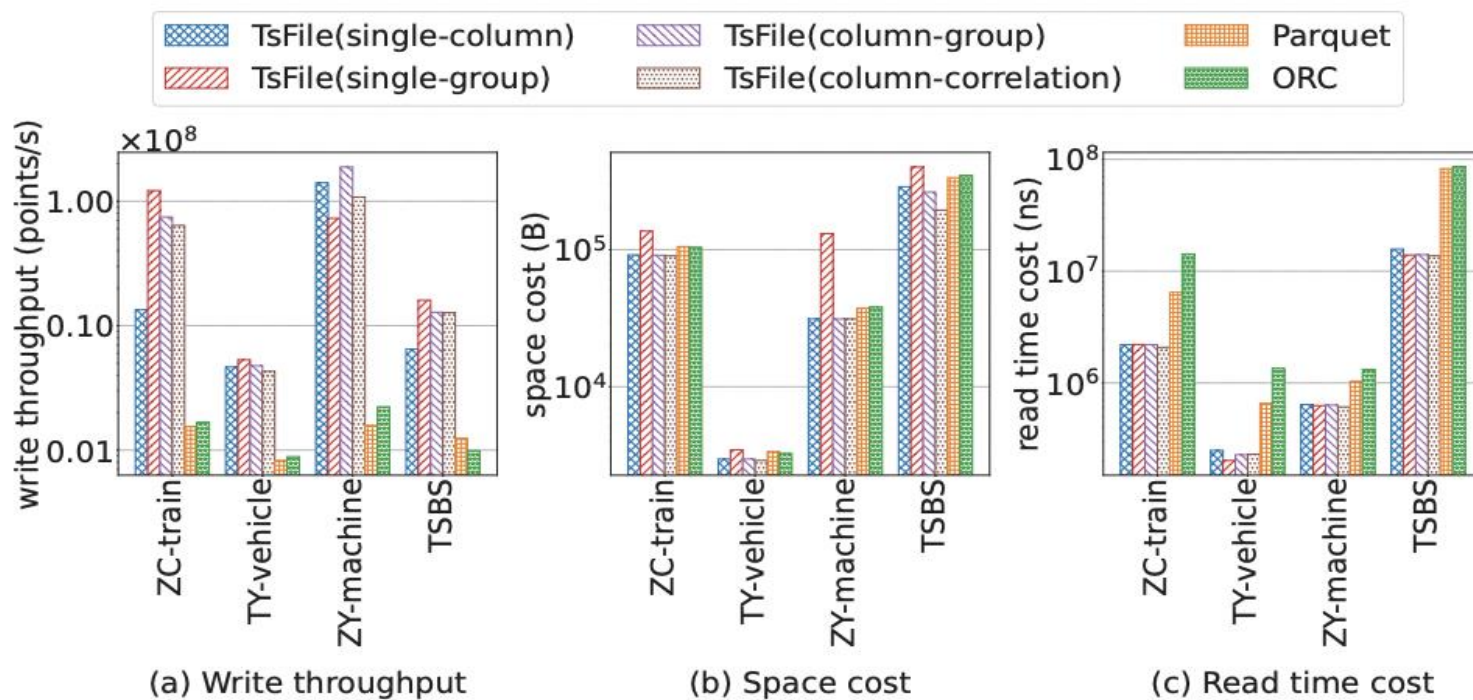
TY-vehicleデータは、建設機械メーカーである天元テクノロジー社の掘削機や配送車両から収集されました。センサーは車速、燃料計、加速度、その他1,033種類の測定値を監視しています。時系列の周波数が不規則なため、データは非常に疎（スパース）です。時系列を整列させると、有効なデータはわずか4.79%です。

ZC-train

ZC-trainデータは、中国中車（**CRRC**）社が製造した上海地下鉄の列車を監視して収集されたものです。温度、速度、荷重など、4,705種類の測定値が含まれています。列車から一括でデータが収集・アップロードされるため、このデータセットは比較的密であり、72.8%が非**NULL**（有効）データです。

ZY-machine

ZY-machineデータは、中国タバコ社の製造機械の生産ログです。水分量、バッチ番号、ロット番号など117種類の指標が記録され、生産状況を監視しています。異なる種類の機械が同時に稼働するため、データは非常に疎であり、88.5%が**NULL**値です。



TsFile(single-column)：単一カラム単位でデータを保存する形式

TsFile(column-group)：カラムグループ単位で保存する形式

TsFile(single-group)：単一グループ単位で保存する形式

TsFile(column-correlation)：カラム間の相関を考慮して保存する形式

最初の図では、Apache IoTDBにおける様々なデータ保存フォーマット（TsFileのバリエーション、Parquet、ORCなど）に対しての書き込みスループット（write throughput）、保存スペースのコスト（space cost）、および読み取り時間のコスト（read time cost）が示されています

主に以下の4種類のクエリタイプを考慮しています 1/2

Q1 – 時間範囲クエリ (Time range query)

あるデバイスから、指定された時間範囲内のデータを取得するクエリです

select series from device where time > ? and time < ?

- 指定した開始時刻と終了時刻の間に記録されたセンサーデータを抽出します

Q2 – 整列クエリ (Alignment query)

複数の系列データ（センサー値など）を同時に整列して取得するクエリです

select series1, series2, ... from device 8

- 同じタイムスタンプ上で、複数のセンサーからの値を並べて取得します

主に以下の4種類のクエリタイプを考慮しています 2/2

Q3 – 集約クエリ (Aggregation query)

- ある系列に対して集計操作（例：カウント）を行うクエリです。

```
select count(series) from device
```

特定センサーの記録回数を数えます

Q4 – ダウンサンプリングクエリ (Downsampling query)

- 時間を一定間隔に分割し、その間隔ごとに集計するクエリです

```
select count(series) from device group by ([start_time, end_time), sampling_interval)
```

1時間ごとに記録数を集計して、データを簡略化します

書き込み性能（Write Performance）

図の結果から分かるように、**TsFile** は他のすべてのファイル形式よりも優れた書き込み性能を示しています。

これは、IoTに特化した構造設計により、`deviceId` や `measurementId` といった冗長な情報を保存しないためです。

TsFile と **ORC** は、通常小さなバッチでストリーミングされるIoTデータの取り込みに適したベクトル化書き込みインターフェース（**vectorized write interface**）を提供しています。

このようなインターフェースは、メモリ局所性（**memory locality**）を向上させ、より高い性能を実現します。

Parquet は最初に 辞書エンコーディング（**dictionary encoding**）を使用しますが、もし辞書やエンコードされたデータのサイズが元のデータサイズを超える場合は、**RLE** など別のエンコーディングに切り替えます。

→ このため、**Parquet** は同じデータを時々**2回**書き込むことになり、結果としてスループットが低くなります。

ORC は 赤黒木（**red-black tree**）に基づく辞書エンコーディングを使用しており、
→ 計算コストが高く、パフォーマンスが低下します。

ストレージコスト (Space Cost)

ストレージコストの結果は図bに示されています。

TsFile は他のフォーマットと比較しても、互換性のある高いストレージ効率を持っています

TsFile は `deviceId` や `measurementId` などの重複する情報をデータカラムとして保存せず、その代わりにより細かいインデックス（索引）を構築します。

これにより若干の追加ストレージを消費しますが、後述する実験結果から明らかなように、クエリ時間（検索速度）において非常に大きな改善をもたらすため、
→ このコストは十分に価値があると言えます。

Parquet や **ORC** では、`deviceId` や `measurementId` の重複データを辞書エンコーディング（dictionary encoding）によってある程度削減することが可能です。しかし、ユニークな ID の数が非常に多い場合（特に **TSBS** データセットでは）、それらの ID は依然として多くのストレージを占有するため、
→ **Parquet** や **ORC** はこのようなケースではあまり適していないと言えます。

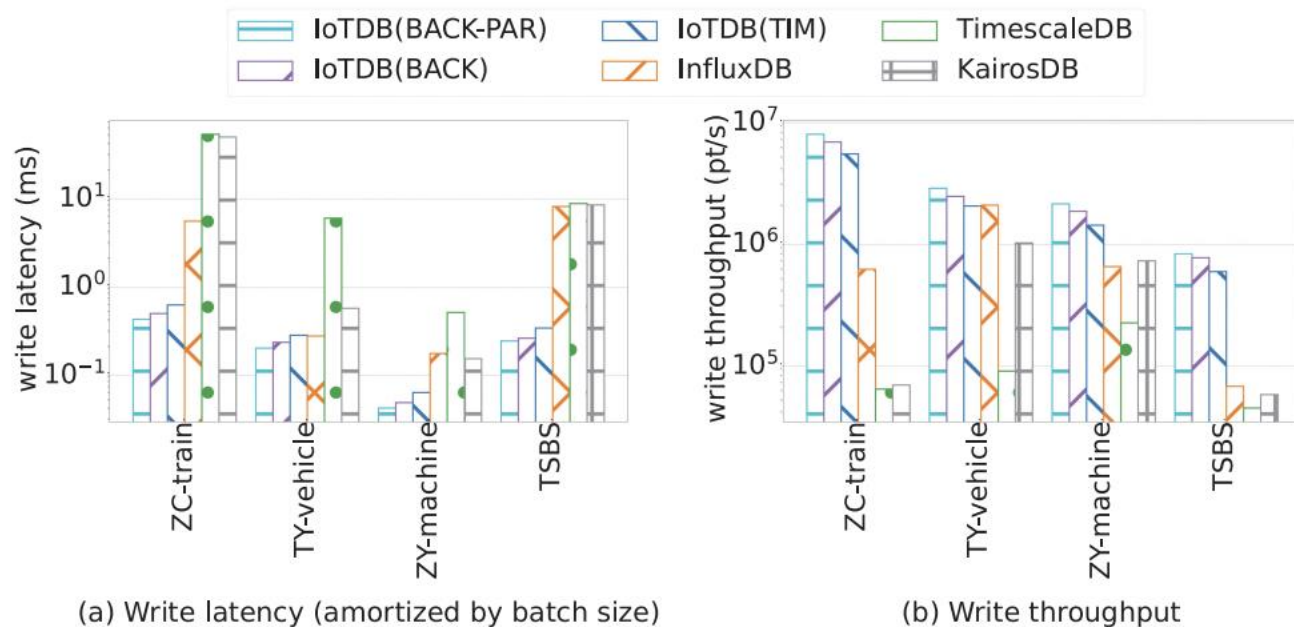
読み取り性能（Read Performance）

図に示されているように、TsFile の読み取り性能はすべてのテストにおいて他の競合フォーマットよりも明らかに優れています。

図に示されているように、TsFile の読み取り性能はすべてのテストにおいて他の競合フォーマットよりも明らかに優れています。

この結果は、TsFile に格納されているインデックスや統計情報を考慮すれば、驚くべきことではありません。
このような大幅に低い読み取り時間コストは、
→ 時系列データに特化したファイルフォーマットを開発する必要性を改めて証明しています。

ORC は 赤黒木（red-black tree）に基づく辞書エンコーディングを使用しており、
→ 計算コストが高く、パフォーマンスが低下します。



IoTDB を InfluxDB、TimescaleDB、および KairosDB と比較し、書き込み性能およびクエリ性能の観点から評価を行いました。



influxdata®

InfluxDB では、時系列データを 1つの measurement（計測値）と 1つの tag（タグ）として保存します。
 → デフォルトの保持ポリシー（retention policy）は無制限の保存期間で、
 → これにより、1週間単位のシャード（shard）が生成されます

KairosDB Logo



TimescaleDB では、次のような手順で構成しています：

1.時刻（time）、シリーズID（series ID）、およびメトリック列（metrics）を含む通常のテーブルを作成します。



Timescale

書き込み性能 (Write Performance)

図2(a)および図2(b)は、それぞれ書き込みレイテンシとスループットを示しています。

全体的に、IoTDBはほぼすべてのテストにおいて優れた性能を示しており、

→ より高い書き込みスループット、

→ より低い書き込みレイテンシを実現しています。

しかし、図20におけるTY-vehicleデータセットでは、IoTDBとInfluxDBの間に明確な差は見られません。

その理由は、このデータセットに文字列 (string) データが多く含まれており、

→ 両システムともにこの種のデータに特化していないため、

→ 性能差が小さくなっているためです。

IoTDBおよびInfluxDB はどちらも 時系列データに最適化されたストレージ実装 を採用しているため、

→ TimescaleDB (PostgreSQLベース) や KairosDB (Cassandraベース) と比較して、

→ 時系列データの取り込みにより適していると言えます。

ソート評価 (Sorting Evaluation)

遅延到着データに対する後方ソート
(backward sort) 手法について、
図に示すような実験を行いました。

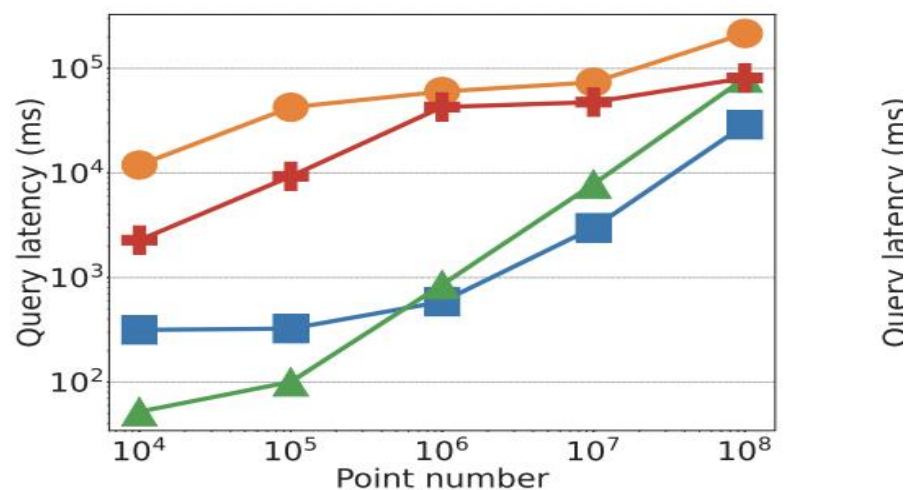
IoTDB(BACK) : 提案する後方ソート方式

IoTDB(BACK-PAR) : その並列最適化バージョン

IoTDB(TIM) : TIMソート に基づく従来の方式

IoTDB(BACK) はベースライン
の IoTDB(TIM) よりも
→ 書き込みスループットが
高く、レイテンシが低いこ
とが確認されました。
さらに、並列最適化を施し
た IoTDB(BACK-PAR) は
→ パフォーマンスを10～
20% 追加で向上させること
ができました。

Time Range Query



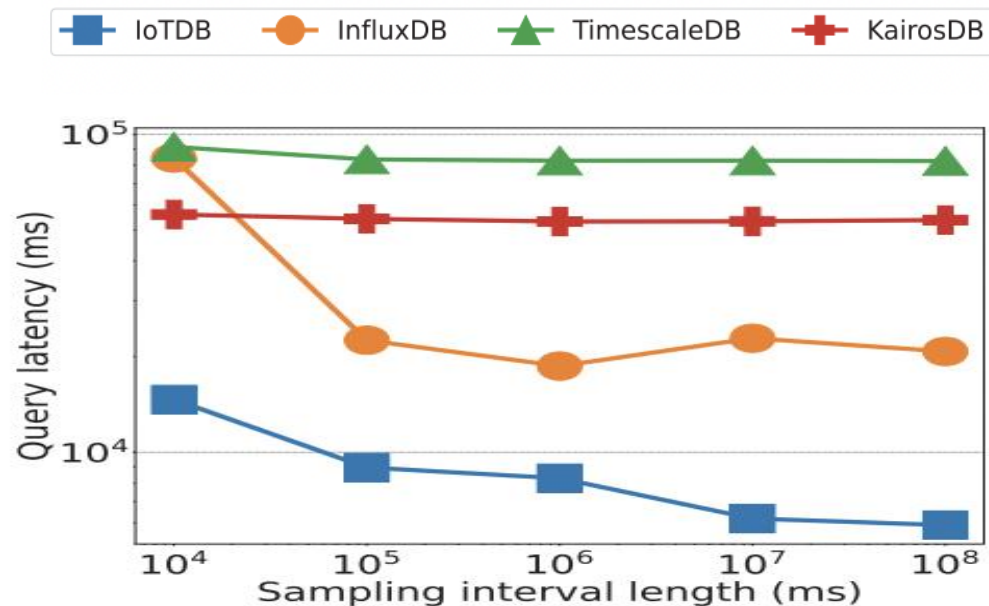
(a) Range query

図に示されているように、クエリ対象の時間範囲が小さい場合、IoTDBのクエリ遅延は他と比べて高くなります。

これは、IoTDBの I/O 単位がチャンク (chunk) であり、デフォルトのチャンクサイズが 64KB 以上に設定されているためです。

そのため、クエリで要求される時間範囲が小さくても、IoTDBはチャンク全体を読み込み、データページを展開する必要があるため、本来必要とされる以上のデータを読み込むことになります。

それでも、読み込むデータが多い割にクエリ遅延は許容範囲内に収まっています。



(d) Downsampling query

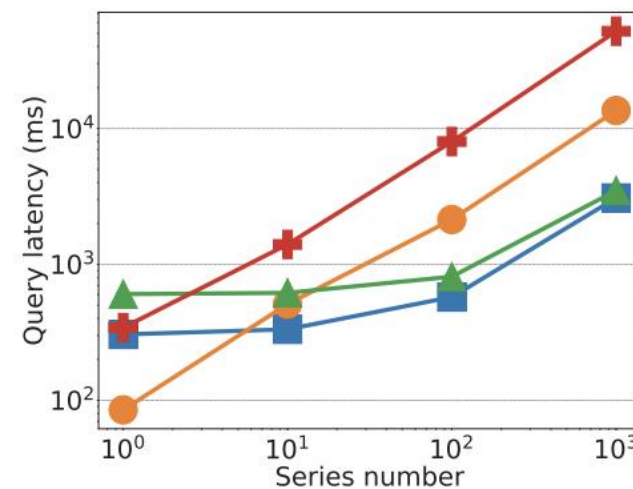
ダウンサンプリングクエリ (Downsampling Query)

IoTDBは、ページレベル、チャンクレベル、ファイルレベルといった複数の統計情報を保持しています。

そのため、サンプリング間隔が大きくなるにつれて、より上位レベルの統計情報を利用してクエリを処理できるようになります。

その結果、ディスクからチャンクを読み込んだり、ページをデコードしたりする必要がなくなり、遅延が大幅に削減されます (図(d) 参照)。

図(b)において、クエリ対象のセンサ数が少ない場合、IoTDBの性能向上はあまり見られません。これは、各時系列に対して新しいスレッドを起動する必要があるためであり、この手法は 多数の系列がクエリ対象となる場合に効果的 です。



(b) Alignment query

機械学習クエリ (Machine Learning Queries)

ARモデルに加えて、ARIMAモデルに対するクエリも評価し、IoTDBがさまざまな種類の機械学習クエリ最適化をサポートできることを示します。

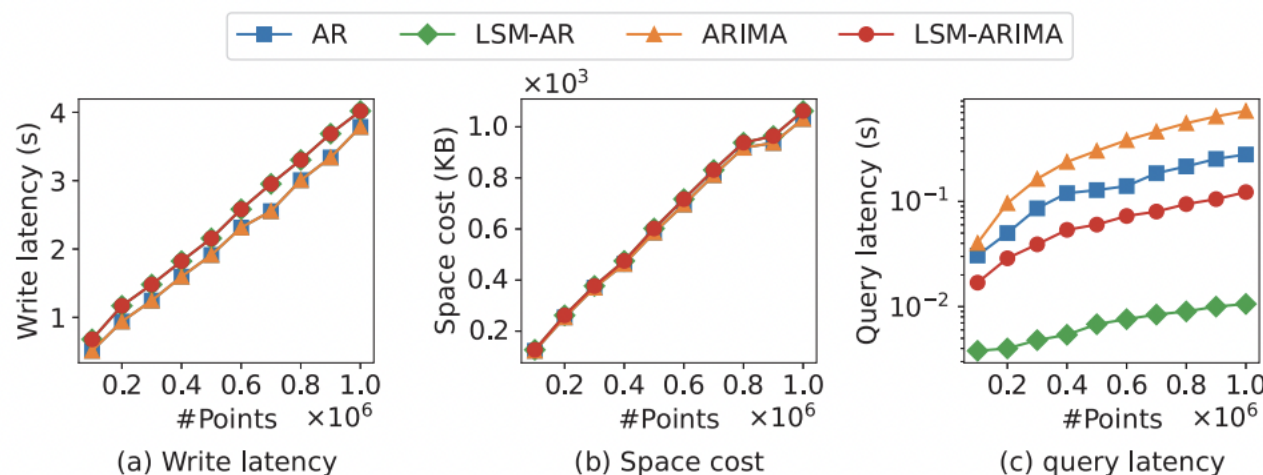


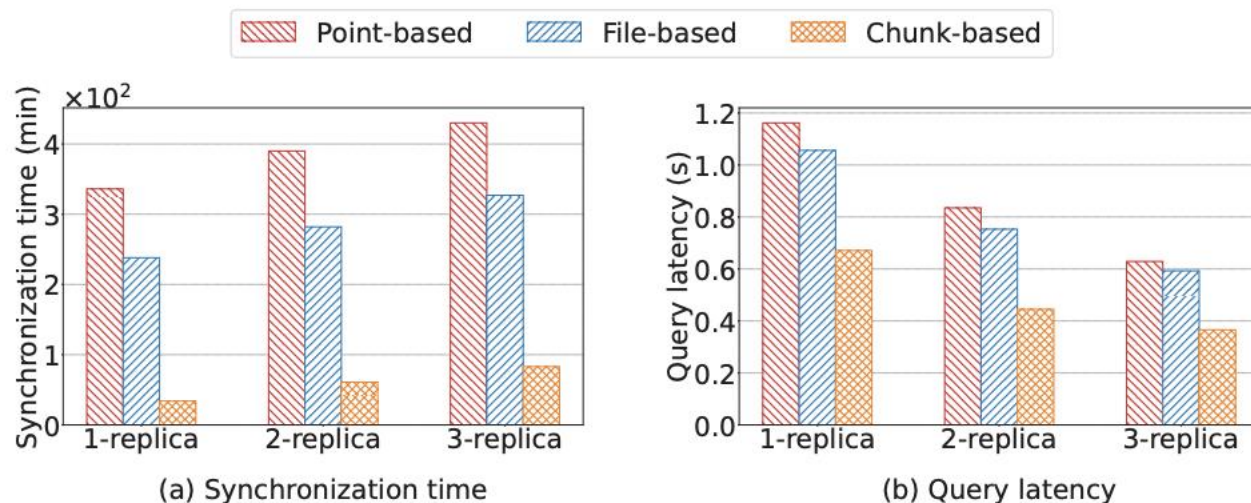
Fig. 22. In-database machine learning query evaluation with AR and ARIMA models.

実験結果によると：

- 書き込みレイテンシは**10%未満**の増加にとどまり、
- 追加のストレージコストはほとんどありません。
- 一方で、高度な**LSM-AR**および**LSM-ARIMA**のクエリレイテンシは、従来の**AR**および**ARIMA**クエリと比較して、約**10分の1**に短縮されました。

図(b)においてデータポイント数がおおよそ 0.9×10^6 付近では、ストレージの増加が緩やかになっていることがわかります。これは、この範囲のデータに重複が多く含まれているためであり、より高い圧縮効率が得られた結果です。

Advanced TsFile Synchronization



図(a)に示されているように、細粒度のチャンクベースのTsFile同期方式は、ファイルベースの転送を行う従来手法と比較して、最大で10倍の時間短縮を実現しています。インデックスのスキップ、チャンクのグルーピングおよび圧縮によって帯域幅の使用が削減され、同期時間が大幅に短縮されました。

チャンクベースのTsFile同期は、図(b)に示すように、クエリ性能を約30%向上させる結果も得られました。この利点は、TsFile同期中に同一の時系列チャンクを再構成することでデータ局所性（data locality）が向上することによって生まれる副次的な効果です

File storageとIoTに適したformat

big data向けのフォーマットはIoTに不向き

- Parquet、Avro、ORCFileなどはIoT向けに設計されていない
- IoTに特有の時系列データや動的なスキーマを考慮していない

ParquetおよびORCの弱点

- IDを通常の列として保存 → 冗長なデータが発生
- Dictionary encodingの制限あり（1MBまで）
- 全メタデータの読み込みが必要 → 探索時間は $O(n)$

TsFileの利点

- deviceIdやmeasurementIdをメタデータに保存 → 保存スペースを節約
- seriesIdのインデックスにより高速なアクセスが可能
- Parquetと比較して書き込みスループットが高く、保存コストが低い

TsFileのパフォーマンス

- ツリー構造のメタデータインデックス → 探索時間は $O(\log n)$
- 図18より → TsFileは最速の読み取り性能を示す

損失圧縮技術との比較（TVStore、MOST、BUFF）

- 上記は損失圧縮を使用
- TsFileはロスレス圧縮で保存コストがさらに低いBUFFよりも少ないストレージを実現

Time Series Database

IoTに適したデータベースの選択

- InfluxDB、TimescaleDB、KairosDBなどの実運用で使われているDBを中心に比較。
- Gorilla、Beringei、ByteSeries、Monarchなどのインメモリ型は、製造業ですべての履歴データを保存するにはコストが高すぎる。
- ModelarDB、MostDBなどのモデルベース型は精度が損なわれる (lossy)**ため、産業用途の厳密なモニタリングには不向き。

InfluxDB vs IoTDB

- InfluxDBはTSM-treeを使用するが、チャンク単位の統計情報しか持たない。
- IoTDBは遅延データの分離技術を導入し、ファイル単位でデバイス・センサー階層の統計情報を保持 → 大規模データにおいて高速なクエリを実現。

TsFileのパフォーマンス

- ツリー構造のメタデータインデックス → 探索時間は $O(\log n)$
- 図18より → TsFileは最速の読み取り性能を示す

損失圧縮技術との比較 (TVStore、MOST、BUFF)

- 上記は損失圧縮を使用
- TsFileはロスレス圧縮で保存コストがさらに低いBUFFよりも少ないストレージを実現

TimescaleDB

- PostgreSQLをベースとした拡張モジュールで、完全なSQL対応。
- ただし、1テーブルあたり1600列の制限があり、多数のセンサーを持つIoT環境には不十分。
- 行ベースの構造のためアライメントクエリには適しているが、クエリコストは行数に比例して増加。
- 図21(b)では、大量のシリーズがある場合に2番目に良い性能を示す。

KairosDB

- NoSQLのCassandra上で動作し、REST API経由でJSON形式のデータを送信 → 解析に時間がかかる。
- IoTDBはinsertTablet()を用いて書き込み効率を向上。
- 図20より、IoTDBはKairosDBよりも優れた書き込み性能を示す。
- また、KairosDBは書き込みを非同期で処理するため、データ損失のリスクがある。

まとめ

本論文では、IoTアプリケーション向けにリアルタイムクエリとビッグデータ分析の両方をサポートするために特別に設計された、オープンアーキテクチャの新しい時系列管理システム **Apache IoTDB** を提案しました。

本システムには、時間と値をカラム型で格納する新しい時系列ファイル形式「**TsFile**」が含まれており、**NULL**値を回避し、効果的な圧縮を実現しています。特筆すべきは、高価な**ETL**処理を必要とせずに、同じ**TsFile**をエンドデバイスで書き込み、エッジサーバのデータベースでクエリし、クラウドのビッグデータシステムで分析することが可能である点です。

数百万の時系列を管理

テラバイト級（**TB**）のデータを処理

毎秒1,000万ポイントの挿入

1日分（10万ポイント）のデータ選択を100ミリ秒以内に実行

3年間（1,000万ポイント超）の集計を100ミリ秒以内に実行可能